



THE UNIVERSITY *of* EDINBURGH
Edinburgh Medical School

Master of Data Science for Health and Social Care (online)

Dissertation

2026

B248593

***Detecting Movement Imbalances using custom
TensorFlow Lite Models utilizing the Google
MediaPipe framework.***

25000

Table of contents

1	Contribution	3
2	Abstract	4
2.1	Aim & objectives	4
2.2	Design	4
2.3	Methods	4
2.4	Results	4
2.5	Conclusion	4
3	Introduction	5
3.1	Research question	6
3.2	Aim	6
3.3	Objectives	6
3.4	Dissertation structure	6
4	Literature Review	7
4.1	Market research	7
4.2	Current Research on Convolutional Neural Networks	8
4.3	Summary	8
5	Methodology	9
5.1	Study design and setting	9
5.2	Data source and characteristics	9
5.3	Variables, features, and preprocessing	9
5.4	Model development and transfer learning	11
5.5	Analysis and performance evaluation	11
6	Implementation	12
6.1	Research setup	12
6.2	Training pipeline and data engineering	13
6.3	Inference pipeline, containerization and CI/CD	13
7	Results	16
7.1	Dataset and preprocessing outcomes	16
7.2	2-Class Training	16
7.2.1	Model performance - 2-class problem	16
7.2.1.1	Training 2-class problem 1	16
7.2.1.2	Training 2-class problem 2	18
7.2.1.3	Training 2-class problem 3 (extra body landmark overlay image-set)	19
7.2.1.4	Error analysis	21
7.3	3-Class Training	21
7.3.1	Model performance - 3-class problem	21
7.3.1.1	Training 1 (EfficientNet-Lite0 architecture)	22
7.3.1.2	Training 2 (EfficientNet-Lite2 architecture)	23
7.3.1.3	Training 3 (EfficientNet-Lite4 architecture)	24
7.3.1.4	Training 4 (MobileNet-V2 architecture)	26
7.3.2	Error analysis	27
7.3.2.1	Comparison of results to 2-class training	28

8	Discussion	29
8.1	Interpretation of findings	29
8.2	Strengths and limitations	29
8.3	Clinical and practical implications	29
9	Conclusion and Future Work	30
10	Appendix A. Figures	31
	References	32

1 Contribution

The development of this dissertation incorporated the selective use of generative artificial intelligence (AI) tools as supportive research and technical assistance instruments. GitHub Copilot and Posit Assistant were utilized to assist in troubleshooting Python dependency conflicts, resolving CI/CD configuration issues involving Jenkins and Docker, and supporting administrative setup tasks related to RStudio Server. In addition, Perplexity was used for targeted literature discovery and contextual identification of relevant academic sources and references. All critical analysis, interpretation, methodological decisions, and written academic content were independently evaluated, verified, and finalized by the author.

Also, I received guidance and support from my two supervisor, Ahmar Shar and John Wilson, who gave me many recommendations during the whole process to conduct the project as well reviewed the draft of the paper once and gave me valuable feedback.

2 Abstract

2.1 Aim & objectives

Finetuning and evaluation of a deep learning model for posture and movement quality assessment that detects and interprets imbalances in the head, neck torso to finetune an pre-trained Google MediaPipe image classification model for posture and movement imbalance detection; and to evaluate the performance of the finetuned classification model.

2.2 Design

Quantitative, observational secondary data analysis evaluating transfer learning on lightweight convolutional neural networks (CNNs) within the Google MediaPipe/TensorFlow Lite ecosystem for static posture classification. Through comparing multiple compact model architectures and deployment-friendly pipelines to assess feasibility for low-cost, on-device, low-latency clinical screening. Model performance is measured on a test set to gauge discrimination across three posture classes and also a binary reformulation of the training was conducted.

2.3 Methods

A curated Kaggle side-view posture image set (n=300) was manually labelled into three classes (normal, slumped, tensed) and expanded via deterministic augmentation to yield around 1900 training images with validation/test splits. Transfer learning fine-tuned classification layers on EfficientNet-Lite0/2/4 and MobileNet-V2 backbones; preprocessing included resizing/normalization and clinically conservative augmentations (small rotations/crops, brightness/contrast jitter, keypoint overlays). Models were exported fully as well as quantized TFLite artifacts and evaluated using accuracy, precision, recall, F1, confusion matrices, and balanced accuracy. A binary “normal vs non-normal” task was also trained to probe clinical interpretability and potential performance gains.

2.4 Results

Across three-class experiments, model architecture EfficientNet-Lite4 achieved the highest test accuracy (41.2%; balanced accuracy 41.7%), followed by EfficientNet-Lite2 (39.2%; 39.8%). Confusion patterns showed over-prediction of “slumped” and frequent misclassification of “normal,” consistent with class overlap and subtle boundary cases. Reformulating as a binary task improved accuracy to 55.9% (with landmark overlays; 56.2% balanced) and 53.2% (without), indicating modest gains and clearer clinical framing, though still below targets for deployment.

2.5 Conclusion

Transfer learning on compact CNNs within the MediaPipe/TFLite stack is feasible for posture imbalance screening, but current performance is limited by small sample size, class ambiguity, and domain subtlety. Targeted data growth (especially boundary and “normal” exemplars), refined labels, posture-aware augmentation, calibration/thresholding, and fairness checks are recommended. With these improvements and expanded validation, a lightweight, on-device tool could support objective posture assessment alongside clinical judgement.

3 Introduction

Healthcare is increasingly recognizing the importance of movement quality and postural balance in preventing injury, managing chronic conditions, and optimizing physical performance (Whittaker et al., 2017). Wijekulasuriya et al. (2025) found in an extensive actual meta analysis that traditional methods for assessing posture and movement often rely on subjective clinical observations, limiting their accessibility and consistency. Advances in artificial intelligence (AI) and computer vision offer promising avenues for developing such objective tools that are scalable and run on consumer hardware that is widely available (Bazarevsky & Grishchenko, 2020; Google, 2024). This dissertation explores the development of Convolutional Neural Networks (CNNs) through transfer learning within the Google MediaPipe framework to detect and analyze head-neck-torso alignment.

Human posture and coordination are central to both health and performance. Impairments such as muscular tension, poor motor control, or postural imbalance contribute significantly to discomfort, injury, and reduced physical performance in daily and professional life (Kubalek-Schröder & Dehler, 2013). Advanced motor skills rely on precise kinaesthetic differentiation, including fine motor control, proprioceptive sensitivity, accurate force regulation, coordinated timing, and the economy of movement. These elements form the basis of effective movement in sports, rehabilitation, and everyday function (Zaucker, 2010). Recent research highlights that chronic low back pain and other movement system impairments are closely associated with sustained postural deviations and maladaptive movement patterns that often develop gradually and may not be apparent during isolated clinical assessments (Ge et al., 2022; Sahrman et al., 2017).

Traditional approaches to assessing posture and coordination rely on subjective evaluations, clinical observation, expensive motion analysis systems, or extensive practitioner expertise. While commercial software for posture assessment exists, these solutions often require professional oversight and remain limited in their ability to capture subtle or habitual patterns of imbalance consistently (Badhe & Kulkarni, 2018). This highlights a clear gap between the qualitative nature of clinical assessments and the potential for more objective, data-driven methods enabled by modern machine learning techniques.

Recent advances in computer vision and artificial intelligence provide opportunities to bridge these limitations. Convolutional Neural Networks (CNNs) have demonstrated high accuracy in identifying patterns within image and video data and are increasingly applied in gait analysis, posture detection, and motion tracking (Jiang et al., 2023). Among available frameworks, the Google MediaPipe framework brings a lightweight approach that can be run in the browser or on mobile devices, utilizing a statistical 3D human shape modelling pipeline offered as a trainable and modular deep learning framework. It can accurately represent full-body details to detect pose and posture (Bazarevsky & Grishchenko, 2020; Xu et al., 2020).

As a physiotherapist with focus on movement quality, I found these gaps were not purely academic but deeply practical in my clinical practice. To reach a level of good understanding of movement (patterns) it required years of training and clinical experience to develop. For instance the ability to accurately assess the quality of movement based on key indicators such as the passive rotatory mobility of the fifth thoracic vertebra (Th5) and adjacent vertebrae (Tachihara & Hamada, 2019). The possibility of using lightweight computer vision to complement clinical expertise, enhance rehabilitation outcomes, and support performance improvement is therefore highly compelling.

3.1 Research question

How can transfer learning on a pre-existing TensorFlow Lite model for pose estimation and image classification be used to detect and analyze posture and movement imbalances?

3.2 Aim

Finetuning and evaluation of a deep learning model for posture and movement quality assessment that detects and interprets imbalances in the head, neck torso and to make it suitable for the clinical context.

3.3 Objectives

1. To finetune an pre-trained Google MediaPipe image classification model for posture and movement imbalance classification.
2. To evaluate the performance of the finetuned classification model.

3.4 Dissertation structure

The present work is structured as follows:

The abstract formulates the research question, aim, objectives, and provides a concise summary of the design, methods, results, and conclusions. Where as the introduction outlines the background, rationale, and research question, and defines the aim and objectives of the project.

The “Literature Review” chapter provides a comprehensive literature review of the current state of AI-based posture and movement analysis, with a focus on the capabilities and limitations of existing pose estimation frameworks such as Google MediaPipe.

The “Methodology” chapter outlines the methods used, including the study design, data sources, variable definitions, model development approach, and performance evaluation strategy.

The “Implementation” chapter details the implementation of the training and inference pipelines with a complex CD/CI integration.

The “Result” chapter presents the results of model performance and error analysis and shows the challenges of training a custom CNN model.

The “Discussion” chapter addresses the findings in relation to existing literature, strengths and limitations, and clinical implications.

The “Conclusion” chapter outlines the expected impact and dissemination plans, while

Whereby the final chapter concludes with a summary of contributions and future work directions.

4 Literature Review

Recent validation studies comparing MediaPipe with gold-standard motion capture systems reveal measurable discrepancies in landmark accuracy, particularly for subtle postural features, highlighting the need for domain-specific refinement for clinical use (Kakuta et al., 2022). Here, “subtle” means inappropriate muscular tension with small or no movement of (the) body(-parts). More broadly, current pose estimation models remain limited by sensitivity to noise and 2D–3D estimation gaps, reducing their reliability for detecting the named tensions and asymmetries critical to movement and pose imbalance analysis (Kakuta et al., 2022; Lachance et al., 2023). Fairness and representativeness in pose estimation systems are also important concerns: current Pose Estimation Models (PEMs) exhibit performance disparities across demographic groups, with reduced accuracy for under-represented populations in training datasets (Lachance et al., 2023). Available datasets frequently focus on male and predominantly white participants between the ages of 19 and 50, while under-representing females, older adults, and individuals with darker skin tones, thereby increasing the risk of systematic bias and reduced generalizability in movement and posture analysis.”

Consequently, while MediaPipe Pose serves as a robust foundation for pose detection, in its standard configuration of training it does not inherently capture the fine-grained symmetry metrics, sagittal and coronal balance indicators, or compensatory movement patterns that rely on hyper- and hypo-tension that are essential for clinical postural and even more for movement analysis. Computer-vision-based deep learning markerless systems nonetheless represent an important step towards closing the gap between theoretical research, that can be even a century old like the principle of primary control of human movement by F.M. Alexander (2017), and practical implementation in remote clinical assessment (Ultralytics, 2025; Wagh et al., 2024).

This project uses transfer learning (Kakuta et al., 2022) to adapt a pre-trained Google MediaPipe CNN model, originally trained on extensive image data, to a new small Kaggle dataset but with more relevant and higher-quality images for posture imbalance detection. By utilizing pre-learned feature representations, the model can achieve faster convergence and improved accuracy even with a comparatively smaller or domain-specific dataset.

4.1 Market research

Modern video analysis with the purpose of detecting pose, movement, and specific markers in sports activities has undergone a paradigm shift, moving from subjective clinical intuition to high-fidelity, data-driven kinematic quantification. Commercial ecosystems like RUNRIGHT 3D (2026) leverage dual-camera 3D reconstruction to analyze over 40 biomechanical metrics without intrusive markers, targeting retail efficiency and rapid, evidence-based footwear recommendations. Similarly, RunDNA (2026) employs advanced motion capture to track kinetic and kinematic variables from foot strike to toe-off, providing practitioners with a granular, 3D anatomical breakdown of movement asymmetries. For integrated clinical environments, the TecnoBody Walker View (2026) combines 3D camera technology with a sensorized, load-cell-equipped treadmill to provide real-time biofeedback and synchronized gait analysis. Supplementing these laboratory-grade tools, ViMove2 (2026) utilizes wearable inertial measurement units (IMUs) to provide continuous, longitudinal monitoring of temporal outcomes like ground contact time and cadence, albeit with sensitivity to speed-dependent discrepancies. At the vanguard of performance capture, Move AI (2026) offers sophisticated, markerless motion capture solutions that utilize computer vision to extract high-fidelity skeletal data from multi-camera footage, facilitating biomechanical analysis outside of traditional studio settings.

The overarching advantage of these commercial approaches lies in their ability to translate complex, high-frequency spatial-temporal data into actionable clinical diagnostics. They reduce inter-rater variability, the differences between multiple assessors, and enable objective longitudinal tracking. Such standardization of classification, may improve the reliability of qualitative (and quantitative) movement assessments in evidence-based rehabilitation settings. Conversely, the disadvantages often involve high capital expenditure, the requirement for controlled capture environments, and often a reliance on proprietary pipelines that can be difficult to consistently integrate into existing clinical (analytical) workflows.

When evaluated against image classification models, the advantages of the latter become apparent: low-cost, low-latency identification of specific postural states and movement patterns, albeit at the expense of the quantitative kinematic precision provided by advanced motion-capture and biomechanical analysis systems.

4.2 Current Research on Convolutional Neural Networks

Cao et al. (2024) state that human posture recognition are machine extractions simulating the eye-function and human learning via feature extraction. They divide this recognition into static and dynamic posture detection, whereby latter one extracts continuous frame sequences and this more widely used. Using images for training and for inference, this approach is static posture classification.

4.3 Summary

Lore Ypsum placeholder text: This section will conclude the literature review in a concise way.

5 Methodology

In this work, we will apply a quantitative, observational design to explore the feasibility of using transfer learning on a pre-trained TensorFlow Lite model for posture and movement imbalance detection. The study will involve secondary data analysis of an existing image dataset curated for posture analysis, with a focus on evaluating the performance of a finetuned CNN model within the Google MediaPipe framework.

5.1 Study design and setting

This study is a secondary data analysis exploring postural and movement imbalance detection using artificial intelligence. The project applies a quantitative, observational design, analyzing existing image data through supervised machine learning, specifically Convolutional Neural Networks (CNNs) implemented via TensorFlow Lite within the Google MediaPipe framework. The aim is to evaluate whether lightweight AI models can detect subtle patterns of imbalance in posture and coordination and to adapt them for potential clinical use through transfer learning on a posture-specific dataset.

The study will be conducted remotely using a secondary dataset. All data analysis will be performed on secured infrastructure provided by the University of Edinburgh. No new data collection or participant interaction is planned.

5.2 Data source and characteristics

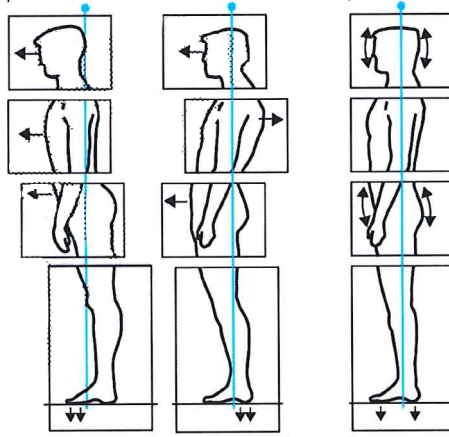
This study uses secondary image data of an openly available “Posture Keypoints Detection – Photos & Labels” dataset on Kaggle. The dataset consists of 300 images in static side view with image resolutions of up to around 1.6 MP and an overall high quality look.

Unlike large-scale, general-purpose image datasets, this Kaggle dataset is specifically curated for posture analysis. It focuses on side-view images suitable for assessing sagittal-plane alignment and upper-body posture. The relatively small dataset size and potential demographic limitations may constrain generalizability and fairness.

5.3 Variables, features, and preprocessing

The posture was classified into 3 types of poses regarding posture classifications by Klein-Vogelbach (1990) as you see in the following illustration:

Figure 1: 3 types of posture by Klein-Vogelbach



The classes

- “tensed posture” (left on the illustration)
- “slumped posture” (middle)
- “normal posture” (right)

The author manually annotated and applied the 300 images to one of these classes according to his clinical experience in the field of orthopaedics and functional movement therapy.

Primary outcome

- Head-neck-torso alignment: Angular relationships and relative positioning of the head, cervical spine, and torso regarding the 3-class model by Klein-Vogelbach.

Exposures/independent variables

- Image category/context: Static posture images in standardized side view.
- Camera/view angle: Minor variation in pose estimation accuracy depending on exact lateral positioning.

Covariates/control features

- Image characteristics: Resolution, background, and lighting, which may influence model performance.
- Posture variation: Degree of anterior head translation, kyphosis, and other sagittal-plane deviations.

Extracted features

- Skeletal keypoints: Annotated landmarks in YOLO pose format and MediaPipe Pose outputs.
- Kinematic proxies: Angular relationships between key joints and segments.
- Postural imbalance markers: Anterior head translation, sagittal imbalance, and deviations from expected alignment patterns.

All selected images from the Kaggle dataset will undergo preprocessing prior to model training, including resizing, normalization, and verification of annotation integrity. Where appropriate, data augmentation (e.g., small rotations, cropping, brightness adjustments) will be applied to improve generalization while preserving clinically meaningful posture characteristics.

5.4 Model development and transfer learning

This study employs a transfer learning approach to develop a domain-specific TensorFlow Lite model suitable for clinical posture and movement analysis. Transfer learning enables the adaptation of pre-trained CNN architectures—such as those used within MediaPipe and trained on large-scale datasets like ImageNet—to the focused task of detecting head-neck-torso imbalances. In practice, base model weights are kept frozen or partially frozen, while classification and selected high-level layers are retrained using the Kaggle posture dataset.

Deploying the resulting model as a quantized TensorFlow Lite artifact supports real-time, on-device inference on mobile and edge devices, reducing latency and preserving user privacy by processing data locally.

5.5 Analysis and performance evaluation

The analysis will proceed in two phases:

1. Descriptive analysis of dataset composition, annotation completeness, and posture-related metrics.
2. AI-based classification and pattern recognition using transfer learning and supervised deep learning methods.

Model performance will be evaluated using accuracy as the primary metric and, where feasible, precision, recall, F1-score, and confusion matrices. Robustness and generalization will be explored through limited distribution-shift experiments (e.g., withheld images with slight background or lighting variation). All hyperparameters, data splits, and preprocessing steps will be documented to ensure reproducibility.

6 Implementation

To utilize the Google MediaPipe framework, extended pipelines and automation were used to prepare the data, train and evaluate models, and produce deployable inference artifacts. The research environment and reproducible steps to build the training pipeline and maintain a production ready data-engineering workflow (including the n8n augmentation workflow) are described.

6.1 Research setup

- **Hardware and runtime**

The development was mostly performed on a remote virtual machine with Ubuntu Linux and specs of 8GB RAM and 4 available CPU Cores. For editing Python scripts, a Jupyter Notebook was used for editing and running the transfer-learning script, for editing the Quarto dissertation project file a RStudio Server. Next to this, a Streamlit WebApp was developed and used to test re-trained models manually. Both the Jupyter notebook and the Streamlit WebApp were deployed via a Docker container on the remote server to manage heavy Python library dependencies of MediaPipe Model Maker (Python 3.10) respective of MediaPipe / TensorFlow Lite (Python 3.13) for the Streamlit app.

- **Software/framework/service stack**

- Python 3.10 (Model Maker) or Python 3.13 (Inference)
- Core CNN libraries: MediaPipe Pose (0.10.20), MediaPipe Model Maker (0.2.1.3), TensorFlow 2.19.1 with TFLite support and Keras (3.12.1)
- Important Python-libraries: OpenCV, NumPy, pandas, Matplotlib, FastAPI (for MediaPipe API), Streamlit (for WebApp)
- Jupyter notebook
- Quarto
- Streamlit (Python WebApp)
- Docker
- Jenkins (CI server)
- n8n (automation/workflows)
- GitHub Actions (dissertation artifact build and deployment)

- **Environment manifests used in this project (congested by Dockerfiles)**

- code/environment.yml
- code/environment_model-maker.yml
- code/requirements.txt
- code/requirements_model-maker.txt

- **Example reproducible local environment:**

```
# MediaPipe Model Maker environment
conda env create -f code/environment_model-maker.yml
conda activate dissertation-3.10

# MediaPipe Pose and TensorFlow Lite environment
conda env create -f code/environment.yml
conda activate dissertation-3.13
```

6.2 Training pipeline and data engineering

Overview - The training pipeline follows an ETL pattern: ingest → validate → augment → split → train → evaluate → archive. Steps are implemented in the repository as Jupyter notebooks and scripts to allow both interactive experimentation and non-interactive CI execution.

Automated augmentation (n8n + MediaPipe) - To expand the original 300 images, the repository uses an n8n workflow to produce duplicates with MediaPipe keypoint overlays and controlled image edits (rotations, crops, brightness adjustments). - The exported n8n workflow is available at `code/n8n/Dissertation_Posture_Analysis.json`. The high-level workflow:

1. Read a list of source images (the initial 300).
2. For each image, call a MediaPipe inference endpoint (either the containerized `code/mediapipe_pose.py` service or an HTTP MediaPipe API) using an HTTP Request node.
3. Receive keypoint coordinates and render overlay images server-side (via the MediaPipe container or a rendering script).
4. Save augmented images to `data/` and record augmentation metadata (source id, parameters).
5. Optionally trigger a downstream training job (Jenkins) or commit augmented data back to the repository index. - This decoupled approach produces auditable augmentation runs and keeps augmentation reproducible.

Training execution - Training can be run locally via `custom_image_classifier_model_training.ipynb` or (remote) inside a container built from `code/Dockerfile` served as Jupyter notebook.

Augmentation & split policy - Augmentations used: small rotations ($\pm 10^\circ$), random crops ($\pm 10\%$), brightness/contrast jitter, and MediaPipe keypoint overlays. Horizontal flips are used when anatomically appropriate. - Splits are deterministic and saved to `data/splits/` for reproducibility.

Artifacts and model storage - Model artifacts (.h5 and .tflite), logs and run figures are stored under `data/models/` and `runs/`. Each run directory includes metadata (git commit, config checksum, timestamp).

6.3 Inference pipeline, containerization and CI/CD

Inference packaging - Inference scripts live in `scripts/` (for example `custom_tflite_image_classifier.py` and `custom_tflite_pose.py`) and operate against quantized .tflite models exported to `data/models/`. - Local inference example:

```
python scripts/custom_tflite_image_classifier.py --model data/models/<run>/model.tflite --in
```

Containerization and compose

- The project maintains two primary Dockerfile artifacts:
- `code/Dockerfile` — training and utilities image.
- `code/Dockerfile_mediapipe`

MediaPipe runtime used for keypoint extraction and overlay rendering. Multi-service development is defined in:

- `code/docker-compose.yml`
- Start development services:

```
docker compose -f code/docker-compose.yml up --build
```

CI/CD

Jenkins is used for build a Jupyter environment on a server to run model training and also builds a Streamlit WebApp for inference testing of previously build models.

GitHub Actions of a GitHub Pages repository is triggered to render the Quarto artifacts:

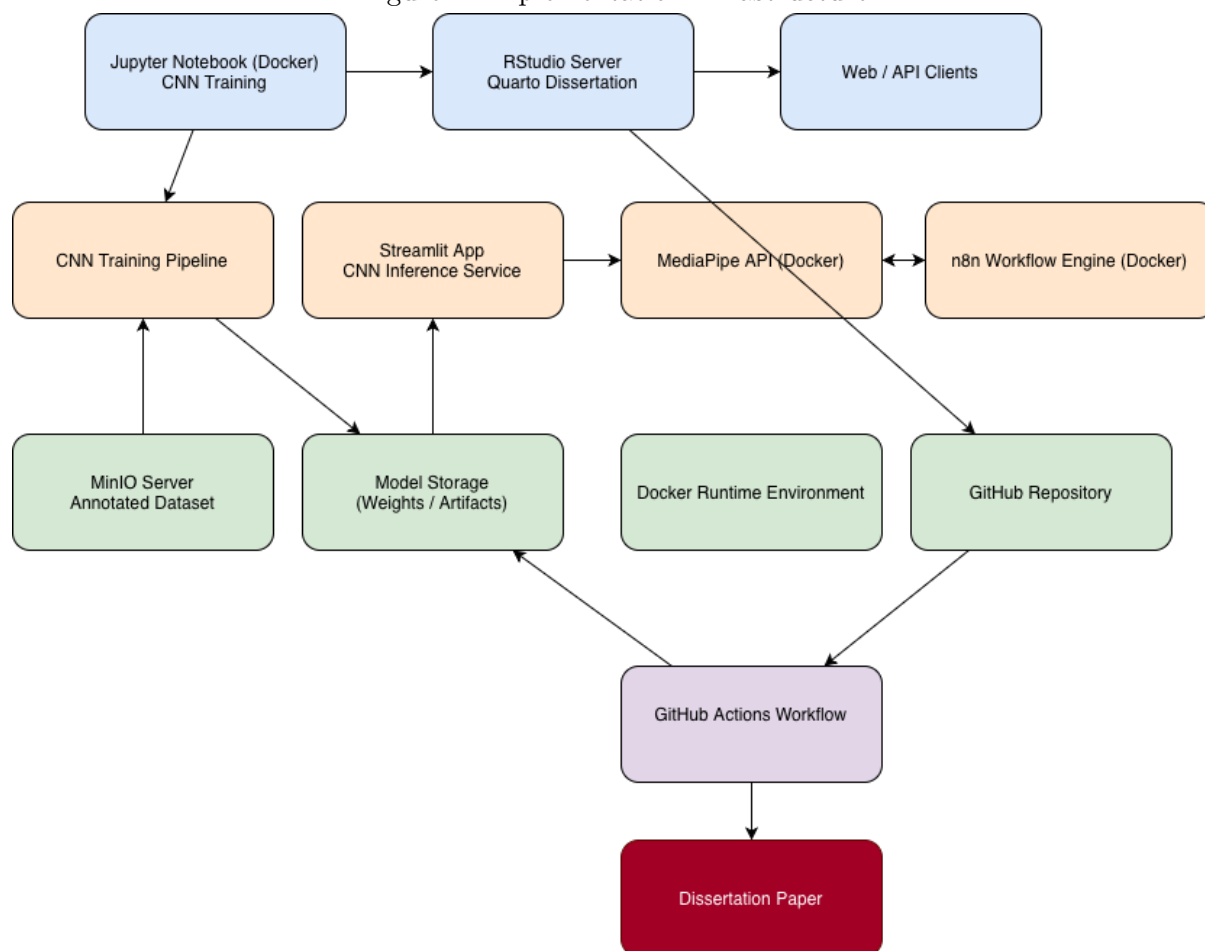
- Checkout repository - Installs Quarto and LaTeX (for PDF rendering).
- Deploys the dissertation project as HTML on <https://drbenjamin.github.io/dissertation.html>

Automation orchestration (n8n)

A **n8n** server run as a containerized service (two container) and to hosts the augmentation workflows. Responsibilities:

- n8n instance (container one)
- Orchestrate MediaPipe API (container two) calls and rendering.
- Persist augmented images and artifacts on a MinIO file storage.

Figure 2: Implementation infrastructure



7 Results

Results were measured for seven model training in total. Three for 2-class and 4 for 3-class image input datasets.

7.1 Dataset and preprocessing outcomes

The dataset and splits used across all runs were consistent. The training set contained 300 original images distributed across two or three classes. To increase the number from 300 original images to counts of ~1500 or ~1800, different augmentation techniques were applied.

- First, the images were duplicated several times and processed via random image editing utilizing the OpenCV framework. For instance, the color-theme or background color were changed, images were cropped or tilted. For each augmented image, 2 changes were applied.
- Second, a subset of duplicated 300 images with body landmarks as overlay were added to specific runs. This was achieved by an existing MediaPipe Pose model which was adding these annotations.

Preprocessing was applied consistently for all runs (same processed image data set) to maintain comparability between 2/3 class runs and architectures.

7.2 2-Class Training

The objective is to reach an accuracy of approximately 80–90%, which is generally considered strong performance for practical detection tasks in applied machine learning settings and for the Google MediaPipe model architecture. Therefore, the images will be annotated into normal and non-normal to define 2 classes.

7.2.1 Model performance - 2-class problem

The same base dataset was used for all three training runs. To address class imbalance in run 2 and 3, data augmentation was applied with an 8× multiplication for normal posture images and a 4× multiplication for non-normal posture images, resulting in a balanced dataset of 1,580 synthetically augmented images.

The third run was extended by an additional 300 images, which were augmented using body landmark overlays generated with the Google MediaPipe Pose framework to incorporate explicit skeletal structure information, which lead to a total of 1880 images.

7.2.1.1 Training 2-class problem 1

Prediction-derived overall accuracy: 0.6000

Model evaluate() loss/accuracy on dataset_obj: 0.546806/0.733333

Analysis batch size: 1

Confusion matrix (rows=true, cols=pred):

True predicted	non-normal	normal
non-normal	16	8
normal	4	2

Row-normalized confusion matrix (recall by class):

True predicted	non-normal	normal
non-normal	0.6667	0.3333
normal	0.6667	0.3333

Per-class support:

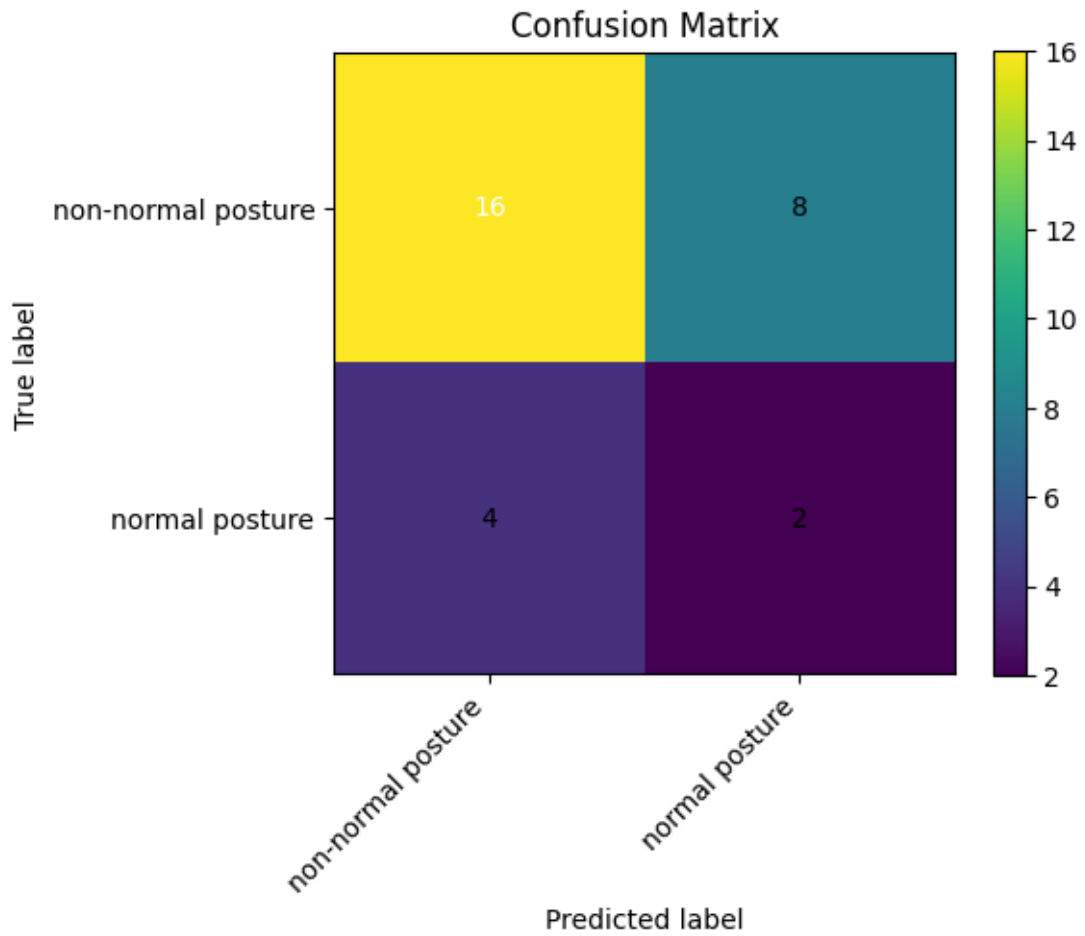
- non-normal posture: 24
- normal posture: 6

Balanced accuracy (macro recall): 0.5000

Classification report:

Class	Precision	Recall	F1-score	Support
non-normal posture	0.8000	0.6667	0.7273	24
normal posture	0.2000	0.3333	0.2500	6
Accuracy			0.6000	30
macro avg	0.5000	0.5000	0.4886	30
weighted avg	0.6800	0.6000	0.6318	30

Figure 3: 2-Class - Training 1



7.2.1.2 Training 2-class problem 2

Prediction-derived overall accuracy: 0.5316

Model evaluate() loss/accuracy on dataset_obj: 0.421809/0.854430

Confusion matrix (rows=true, cols=pred):

True predicted	non-normal	normal
non-normal	35	47
normal	27	49

Row-normalized confusion matrix (recall by class):

True predicted	non-normal	normal
non-normal	0.4268	0.5732
normal	0.3553	0.6447

Per-class support:

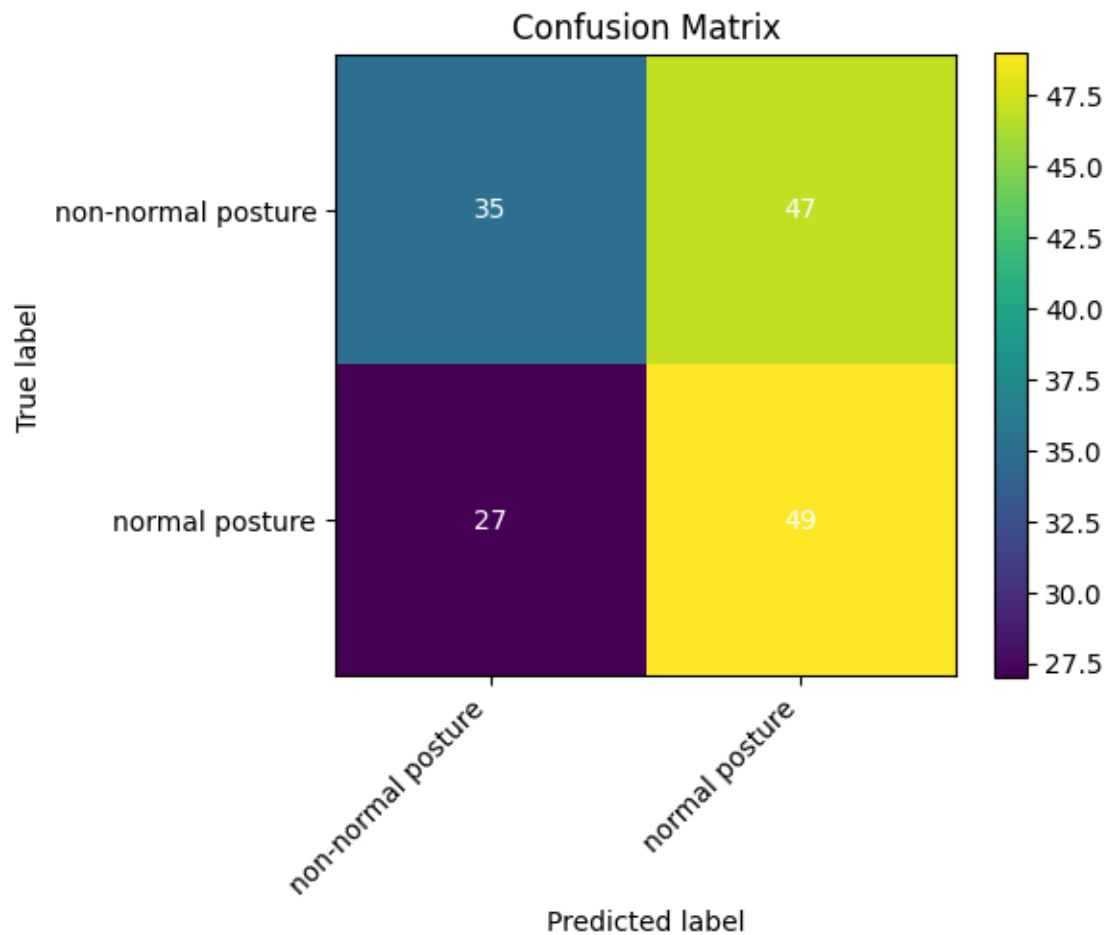
- non-normal posture: 82
- normal posture: 76

Balanced accuracy (macro recall): 0.5358

Classification report:

Class	Precision	Recall	F1-score	Support
non-normal posture	0.5645	0.4268	0.4861	82
normal posture	0.5104	0.6447	0.5698	76
Accuracy			0.5316	158
macro avg	0.5375	0.5358	0.5279	158
weighted avg	0.5385	0.5316	0.5264	158

Figure 4: 2-Class - Training 2



7.2.1.3 Training 2-class problem 3 (extra body landmark overlay image-set)

Prediction-derived overall accuracy: 0.5585

Model evaluate() loss/accuracy on dataset_obj: 0.454466/0.840426

Confusion matrix (rows=true, cols=pred):

True predicted	non-normal	normal
non-normal	44	47
normal	36	61

Row-normalized confusion matrix (recall by class):

True predicted	non-normal	normal
non-normal	0.4835	0.5165
normal	0.3711	0.6289

Per-class support:

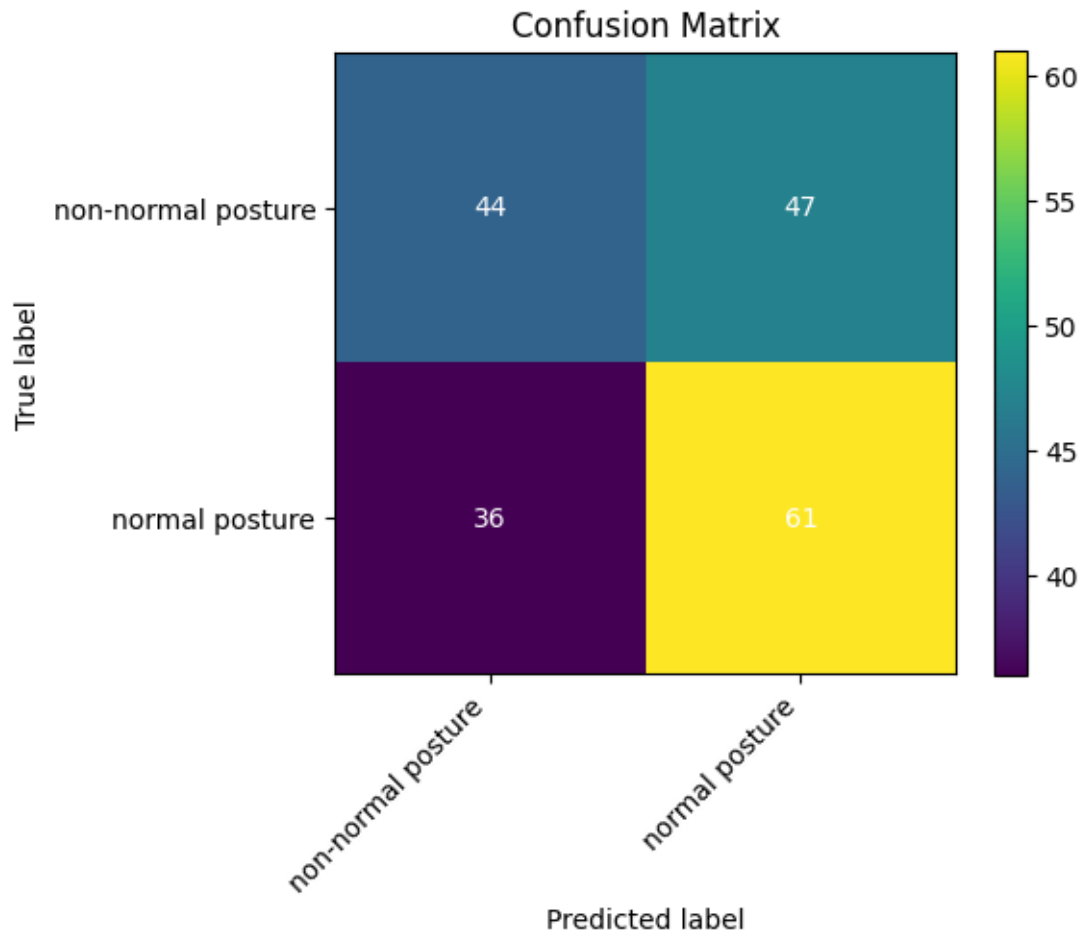
- non-normal posture: 91
- normal posture: 97

Balanced accuracy (macro recall): 0.5562

Classification report:

Class	Precision	Recall	F1-score	Support
non-normal posture	0.5500	0.4835	0.5146	91
normal posture	0.5648	0.6289	0.5951	97
Accuracy			0.5585	188
macro avg	0.5574	0.5562	0.5549	188
weighted avg	0.5576	0.5585	0.5562	188

Figure 5: 2-Class - Training 3



7.2.1.4 Error analysis

Model 1 (no overlays) performs with an accuracy of 0.5316 (balanced 0.5358), where as Model 2 (with landmark overlays) with 0.5585. (balanced 0.5562). Landmark overlays yield a modest overall gain and meaningfully improve sensitivity to non-normal cases, with a small trade-off in normal recall. Both models perform only slightly above the 50% chance baseline.

7.3 3-Class Training

To achieve the pose detection defined by Klein-Vogelbach, 3 classes were defined.

7.3.1 Model performance - 3-class problem

Four training runs were performed using different base architectures. For each run the reported metrics below are derived from evaluation on the held-out test split and from prediction-derived analysis.

7.3.1.1 Training 1 (EfficientNet-Lite0 architecture)

Prediction-derived overall accuracy: 0.3216

Model evaluate(internal performance metric) loss/accuracy on dataset_obj: 0.638240/0.824121

Confusion matrix

True predicted	normal	slumped	tensed
normal	4	38	21
slumped	2	40	20
tensed	3	51	20

Row-normalized confusion matrix (recall by class)

True predicted	normal	slumped	tensed
normal	0.0635	0.6032	0.3333
slumped	0.0323	0.6452	0.3226
tensed	0.0405	0.6892	0.2703

Per-class support:

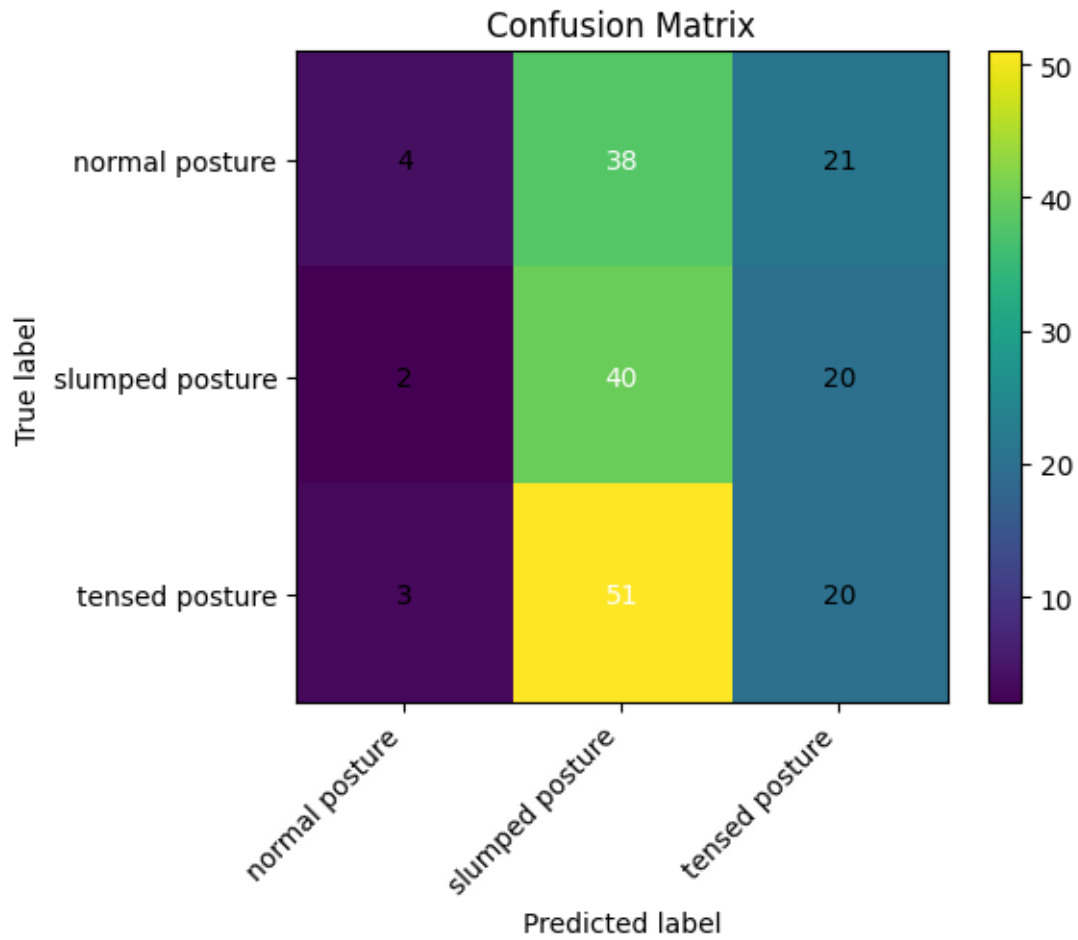
- normal posture: 63
- slumped posture: 62
- tensed posture: 74

Balanced accuracy (macro recall): 0.3263

Classification report

Class	Precision	Recall	F1-score	Support
normal posture	0.4444	0.0635	0.1111	63
slumped posture	0.3101	0.6452	0.4188	62
tensed posture	0.3279	0.2703	0.2963	74
Accuracy			0.3216	199
Macro avg.	0.3608	0.3263	0.2754	199
Weighted avg.	0.3592	0.3216	0.2759	199

Figure 6: 3-Class - Training 1



7.3.1.2 Training 2 (EfficientNet-Lite2 architecture)

Prediction-derived overall accuracy: 0.3920

Model evaluate(internal performance metric) loss/accuracy on dataset_obj: 0.645087/0.849246

Confusion matrix

True predicted	normal	slumped	tensed
normal	16	36	11
slumped	10	39	13
tensed	12	39	23

Row-normalized confusion matrix (recall by class)

True predicted	normal	slumped	tensed
normal	0.2540	0.5714	0.1746
slumped	0.1613	0.6290	0.2097
tensed	0.1622	0.5270	0.3108

Per-class support:

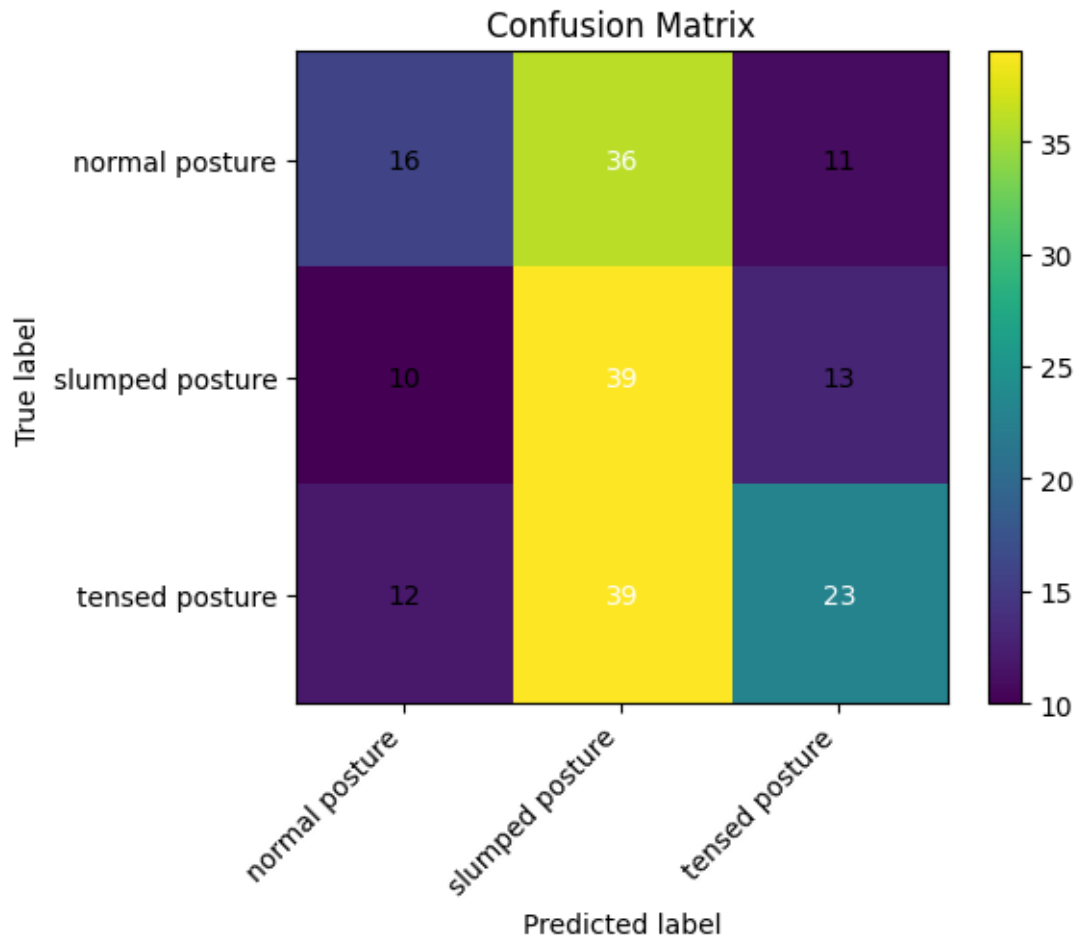
- normal posture: 63
- slumped posture: 62
- tensed posture: 74

Balanced accuracy (macro recall): 0.3979

Classification report

Class	Precision	Recall	F1-score	Support
normal posture	0.4211	0.2540	0.3168	63
slumped posture	0.3421	0.6290	0.4432	62
tensed posture	0.4894	0.3108	0.3802	74
Accuracy			0.3920	199
Macro avg.	0.4175	0.3979	0.3801	199
Weighted avg.	0.4219	0.3920	0.3797	199

Figure 7: 3-Class - Training 2



7.3.1.3 Training 3 (EfficientNet-Lite4 architecture)

Prediction-derived overall accuracy: 0.4121

Model evaluate(internal performance metric) loss/accuracy on dataset_obj: 0.703537/0.758794

Confusion matrix

True predicted	normal	slumped	tensed
normal	18	33	12
slumped	12	38	12
tensed	15	33	26

Row-normalized confusion matrix (recall by class)

True predicted	normal	slumped	tensed
normal	0.2857	0.5238	0.1905
slumped	0.1935	0.6129	0.1935
tensed	0.2027	0.4459	0.3514

Per-class support:

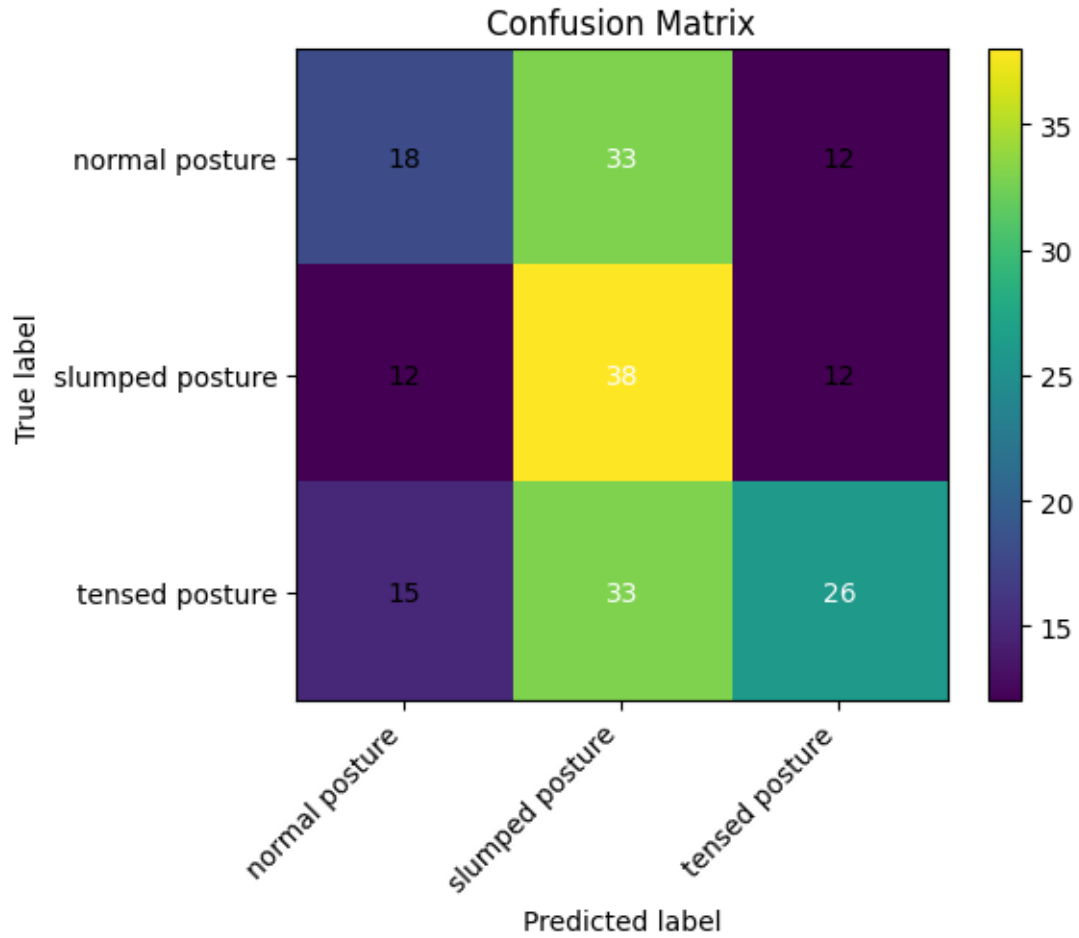
- normal posture: 63
- slumped posture: 62
- tensed posture: 74

Balanced accuracy (macro recall): 0.4167

Classification report

Class	Precision	Recall	F1-score	Support
normal posture	0.4000	0.2857	0.3333	63
slumped posture	0.3654	0.6129	0.4578	62
tensed posture	0.5200	0.3514	0.4194	74
Accuracy			0.4121	199
Macro avg.	0.4285	0.4167	0.4035	199
Weighted avg.	0.4338	0.4121	0.4041	199

Figure 8: 3-Class - Training 3



7.3.1.4 Training 4 (MobileNet-V2 architecture)

Prediction-derived overall accuracy: 0.3065

Model evaluate(internal performance metric) loss/accuracy on dataset_obj: 0.736599/0.748744

Confusion matrix

True predicted	normal	slumped	tensed
normal	24	21	18
slumped	26	20	16
tensed	23	34	17

Row-normalized confusion matrix (recall by class)

True predicted	normal	slumped	tensed
normal	0.3810	0.3333	0.2857
slumped	0.4194	0.3226	0.2581
tensed	0.3108	0.4595	0.2297

Per-class support:

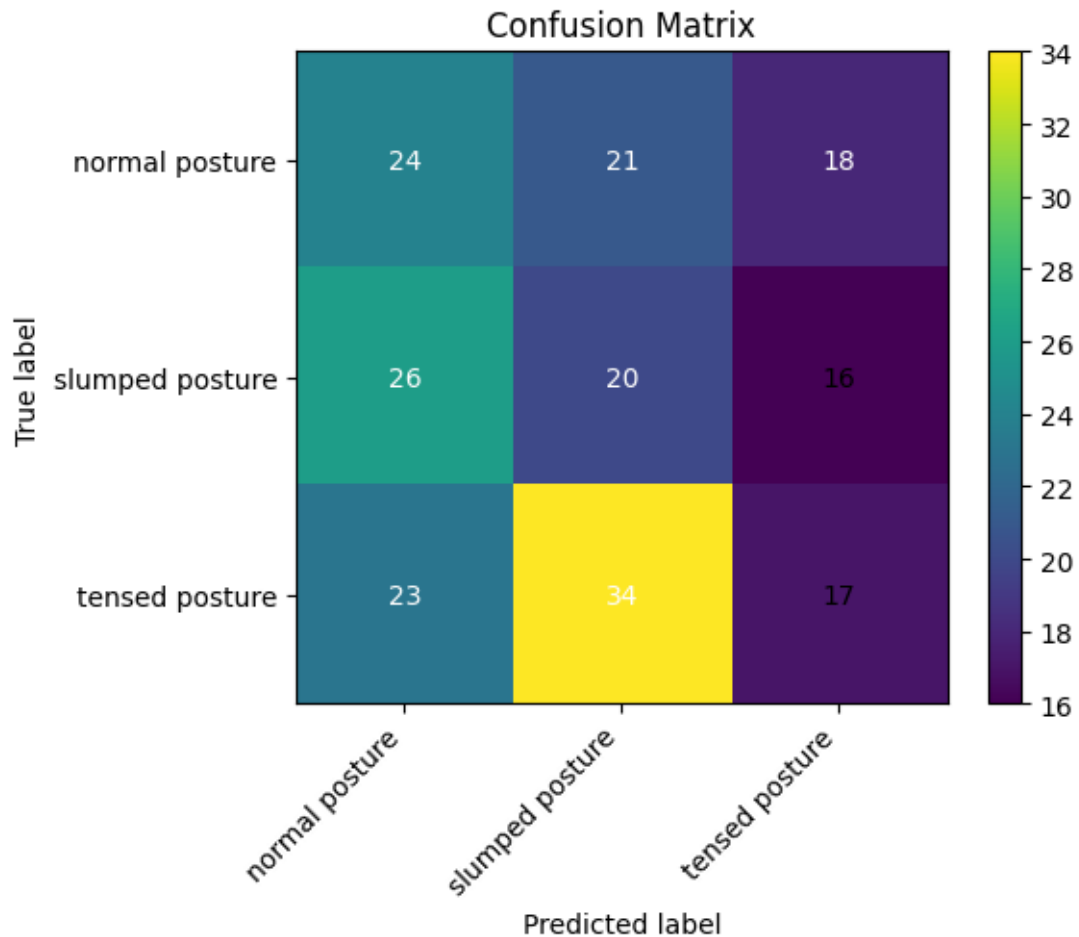
- normal posture: 63
- slumped posture: 62
- tensed posture: 74

Balanced accuracy (macro recall): 0.3111

Classification report

Class	Precision	Recall	F1-score	Support
normal posture	0.3288	0.3810	0.3529	63
slumped posture	0.2667	0.3226	0.2920	62
tensed posture	0.3333	0.2297	0.2720	74
Accuracy			0.3065	199
Macro avg.	0.3096	0.3111	0.3056	199
Weighted avg.	0.3111	0.3065	0.3038	199

Figure 9: 3-Class - Training 4



7.3.2 Error analysis

Across all experiments a consistent pattern of misclassification is apparent. The models show relatively high recall for the **slumped posture** class but lower precision, indicating a tendency

to predict **slumped** for a wide range of inputs. **Normal posture** is frequently misclassified (low recall in several runs), and **tensed posture** predictions vary by architecture: deeper Lite variants tended to improve overall discrimination.

- Best overall accuracy and balanced recall were achieved by EfficientNet-Lite4 (accuracy 0.4121, balanced accuracy 0.4167).
- EfficientNet-Lite2 performed second best (accuracy 0.3920, balanced accuracy 0.3979).
- EfficientNet-Lite0 and MobileNet-V2 produced lower overall accuracy (0.3216 and 0.3065 respectively) and lower balanced accuracy.

Common failure modes observed in confusion matrices:

- Confusion between **normal** and **slumped**: many **normal** examples are predicted as **slumped**, suggesting the model is sensitive to subtle postural variations or that the dataset contains ambiguous examples near class boundaries.
- Over-prediction of **slumped**: reflected in row-normalized confusion matrices where the middle column (predicted **slumped**) is large across runs.
- Lower discrimination for **tensed** vs. other classes in the smaller architectures, improved modestly in larger EfficientNet-Lite variants.

Implications and next steps:

- The results indicate that the models can capture some posture-related features but that class overlap and limited dataset size constrain discriminative performance. Better class balance, additional labelled data for edge cases, and further augmentation tailored to posture variations should be considered.
- Model calibration and threshold tuning (or use of ensemble methods) may improve precision for the **slumped** and **tensed** classes.
- Further work should include qualitative review of misclassified examples to identify systematic annotation or dataset issues and targeted augmentation or re-labelling where necessary.

7.3.2.1 Comparison of results to 2-class training

Chance levels for the 3-class are 1/3 or 33.3%, for the 2-class 1/2 or 50%. Because of this difference, a chance-balanced accuracy will be calculated.

- Best 3-class model: balanced accuracy =0.4167.
 - $\ast = 0.4167 - 1/3 = 0.0834$
 - $\ast = 1 - 1/3 - 0.0834 = 0.6667$
 - $\ast = 0.6667 \times 0.0834 = 0.0556$
 - $\ast = 0.0556 + 0.0834 = 0.1390$
 - $\ast = 0.1390 + 0.0556 = 0.1946$
 - $\ast = 0.1946 + 0.0556 = 0.2502$
 - $\ast = 0.2502 + 0.0556 = 0.3058$
 - $\ast = 0.3058 + 0.0556 = 0.3614$
 - $\ast = 0.3614 + 0.0556 = 0.4170$
 - $\ast = 0.4170 + 0.0556 = 0.4726$
 - $\ast = 0.4726 + 0.0556 = 0.5282$
 - $\ast = 0.5282 + 0.0556 = 0.5838$
 - $\ast = 0.5838 + 0.0556 = 0.6394$
 - $\ast = 0.6394 + 0.0556 = 0.6950$
 - $\ast = 0.6950 + 0.0556 = 0.7506$
 - $\ast = 0.7506 + 0.0556 = 0.8062$
 - $\ast = 0.8062 + 0.0556 = 0.8618$
 - $\ast = 0.8618 + 0.0556 = 0.9174$
 - $\ast = 0.9174 + 0.0556 = 0.9730$
 - $\ast = 0.9730 + 0.0556 = 1.0286$
 - $\ast = 1.0286 + 0.0556 = 1.0842$
 - $\ast = 1.0842 + 0.0556 = 1.1398$
 - $\ast = 1.1398 + 0.0556 = 1.1954$
 - $\ast = 1.1954 + 0.0556 = 1.2510$
 - $\ast = 1.2510 + 0.0556 = 1.3066$
 - $\ast = 1.3066 + 0.0556 = 1.3622$
 - $\ast = 1.3622 + 0.0556 = 1.4178$
 - $\ast = 1.4178 + 0.0556 = 1.4734$
 - $\ast = 1.4734 + 0.0556 = 1.5290$
 - $\ast = 1.5290 + 0.0556 = 1.5846$
 - $\ast = 1.5846 + 0.0556 = 1.6402$
 - $\ast = 1.6402 + 0.0556 = 1.6958$
 - $\ast = 1.6958 + 0.0556 = 1.7514$
 - $\ast = 1.7514 + 0.0556 = 1.8070$
 - $\ast = 1.8070 + 0.0556 = 1.8626$
 - $\ast = 1.8626 + 0.0556 = 1.9182$
 - $\ast = 1.9182 + 0.0556 = 1.9738$
 - $\ast = 1.9738 + 0.0556 = 2.0294$
 - $\ast = 2.0294 + 0.0556 = 2.0850$
 - $\ast = 2.0850 + 0.0556 = 2.1406$
 - $\ast = 2.1406 + 0.0556 = 2.1962$
 - $\ast = 2.1962 + 0.0556 = 2.2518$
 - $\ast = 2.2518 + 0.0556 = 2.3074$
 - $\ast = 2.3074 + 0.0556 = 2.3630$
 - $\ast = 2.3630 + 0.0556 = 2.4186$
 - $\ast = 2.4186 + 0.0556 = 2.4742$
 - $\ast = 2.4742 + 0.0556 = 2.5298$
 - $\ast = 2.5298 + 0.0556 = 2.5854$
 - $\ast = 2.5854 + 0.0556 = 2.6410$
 - $\ast = 2.6410 + 0.0556 = 2.6966$
 - $\ast = 2.6966 + 0.0556 = 2.7522$
 - $\ast = 2.7522 + 0.0556 = 2.8078$
 - $\ast = 2.8078 + 0.0556 = 2.8634$
 - $\ast = 2.8634 + 0.0556 = 2.9190$
 - $\ast = 2.9190 + 0.0556 = 2.9746$
 - $\ast = 2.9746 + 0.0556 = 3.0302$
 - $\ast = 3.0302 + 0.0556 = 3.0858$
 - $\ast = 3.0858 + 0.0556 = 3.1414$
 - $\ast = 3.1414 + 0.0556 = 3.1970$
 - $\ast = 3.1970 + 0.0556 = 3.2526$
 - $\ast = 3.2526 + 0.0556 = 3.3082$
 - $\ast = 3.3082 + 0.0556 = 3.3638$
 - $\ast = 3.3638 + 0.0556 = 3.4194$
 - $\ast = 3.4194 + 0.0556 = 3.4750$
 - $\ast = 3.4750 + 0.0556 = 3.5306$
 - $\ast = 3.5306 + 0.0556 = 3.5862$
 - $\ast = 3.5862 + 0.0556 = 3.6418$
 - $\ast = 3.6418 + 0.0556 = 3.6974$
 - $\ast = 3.6974 + 0.0556 = 3.7530$
 - $\ast = 3.7530 + 0.0556 = 3.8086$
 - $\ast = 3.8086 + 0.0556 = 3.8642$
 - $\ast = 3.8642 + 0.0556 = 3.9198$
 - $\ast = 3.9198 + 0.0556 = 3.9754$
 - $\ast = 3.9754 + 0.0556 = 4.0310$
 - $\ast = 4.0310 + 0.0556 = 4.0866$
 - $\ast = 4.0866 + 0.0556 = 4.1422$
 - $\ast = 4.1422 + 0.0556 = 4.1978$
 - $\ast = 4.1978 + 0.0556 = 4.2534$
 - $\ast = 4.2534 + 0.0556 = 4.3090$
 - $\ast = 4.3090 + 0.0556 = 4.3646$
 - $\ast = 4.3646 + 0.0556 = 4.4202$
 - $\ast = 4.4202 + 0.0556 = 4.4758$
 - $\ast = 4.4758 + 0.0556 = 4.5314$
 - $\ast = 4.5314 + 0.0556 = 4.5870$
 - $\ast = 4.5870 + 0.0556 = 4.6426$
 - $\ast = 4.6426 + 0.0556 = 4.6982$
 - $\ast = 4.6982 + 0.0556 = 4.7538$
 - $\ast = 4.7538 + 0.0556 = 4.8094$
 - $\ast = 4.8094 + 0.0556 = 4.8650$
 - $\ast = 4.8650 + 0.0556 = 4.9206$
 - $\ast = 4.9206 + 0.0556 = 4.9762$
 - $\ast = 4.9762 + 0.0556 = 5.0318$
 - $\ast = 5.0318 + 0.0556 = 5.0874$
 - $\ast = 5.0874 + 0.0556 = 5.1430$
 - $\ast = 5.1430 + 0.0556 = 5.1986$
 - $\ast = 5.1986 + 0.0556 = 5.2542$
 - $\ast = 5.2542 + 0.0556 = 5.3098$
 - $\ast = 5.3098 + 0.0556 = 5.3654$
 - $\ast = 5.3654 + 0.0556 = 5.4210$
 - $\ast = 5.4210 + 0.0556 = 5.4766$
 - $\ast = 5.4766 + 0.0556 = 5.5322$
 - $\ast = 5.5322 + 0.0556 = 5.5878$
 - $\ast = 5.5878 + 0.0556 = 5.6434$
 - $\ast = 5.6434 + 0.0556 = 5.6990$
 - $\ast = 5.6990 + 0.0556 = 5.7546$
 - $\ast = 5.7546 + 0.0556 = 5.8102$
 - $\ast = 5.8102 + 0.0556 = 5.8658$
 - $\ast = 5.8658 + 0.0556 = 5.9214$
 - $\ast = 5.9214 + 0.0556 = 5.9770$
 - $\ast = 5.9770 + 0.0556 = 6.0326$
 - $\ast = 6.0326 + 0.0556 = 6.0882$
 - $\ast = 6.0882 + 0.0556 = 6.1438$
 - $\ast = 6.1438 + 0.0556 = 6.1994$
 - $\ast = 6.1994 + 0.0556 = 6.2550$
 - $\ast = 6.2550 + 0.0556 = 6.3106$
 - $\ast = 6.3106 + 0.0556 = 6.3662$
 - $\ast = 6.3662 + 0.0556 = 6.4218$
 - $\ast = 6.4218 + 0.0556 = 6.4774$
 - $\ast = 6.4774 + 0.0556 = 6.5330$
 - $\ast = 6.5330 + 0.0556 = 6.5886$
 - $\ast = 6.5886 + 0.0556 = 6.6442$
 - $\ast = 6.6442 + 0.0556 = 6.6998$
 - $\ast = 6.6998 + 0.0556 = 6.7554$
 - $\ast = 6.7554 + 0.0556 = 6.8110$
 - $\ast = 6.8110 + 0.0556 = 6.8666$
 - $\ast = 6.8666 + 0.0556 = 6.9222$
 - $\ast = 6.9222 + 0.0556 = 6.9778$
 - $\ast = 6.9778 + 0.0556 = 7.0334$
 - $\ast = 7.0334 + 0.0556 = 7.0890$
 - $\ast = 7.0890 + 0.0556 = 7.1446$
 - $\ast = 7.1446 + 0.0556 = 7.2002$
 - $\ast = 7.2002 + 0.0556 = 7.2558$
 - $\ast = 7.2558 + 0.0556 = 7.3114$
 - $\ast = 7.3114 + 0.0556 = 7.3670$
 - $\ast = 7.3670 + 0.0556 = 7.4226$
 - $\ast = 7.4226 + 0.0556 = 7.4782$
 - $\ast = 7.4782 + 0.0556 = 7.5338$
 - $\ast = 7.5338 + 0.0556 = 7.5894$
 - $\ast = 7.5894 + 0.0556 = 7.6450$
 - $\ast = 7.6450 + 0.0556 = 7.7006$
 - $\ast = 7.7006 + 0.0556 = 7.7562$
 - $\ast = 7.7562 + 0.0556 = 7.8118$
 - $\ast = 7.8118 + 0.0556 = 7.8674$
 - $\ast = 7.8674 + 0.0556 = 7.9230$
 - $\ast = 7.9230 + 0.0556 = 7.9786$
 - $\ast = 7.9786 + 0.0556 = 8.0342$
 - $\ast = 8.0342 + 0.0556 = 8.0898$
 - $\ast = 8.0898 + 0.0556 = 8.1454$
 - $\ast = 8.1454 + 0.0556 = 8.2010$
 - $\ast = 8.2010 + 0.0556 = 8.2566$
 - $\ast = 8.2566 + 0.0556 = 8.3122$
 - $\ast = 8.3122 + 0.0556 = 8.3678$
 - $\ast = 8.3678 + 0.0556 = 8.4234$
 - $\ast = 8.4234 + 0.0556 = 8.4790$
 - $\ast = 8.4790 + 0.0556 = 8.5346$
 - $\ast = 8.5346 + 0.0556 = 8.5902$
 - $\ast = 8.5902 + 0.0556 = 8.6458$
 - $\ast = 8.6458 + 0.0556 = 8.7014$
 - $\ast = 8.7014 + 0.0556 = 8.7570$
 - $\ast = 8.7570 + 0.0556 = 8.8126$
 - $\ast = 8.8126 + 0.0556 = 8.8682$
 - $\ast = 8.8682 + 0.0556 = 8.9238$
 - $\ast = 8.9238 + 0.0556 = 8.9794$
 - $\ast = 8.9794 + 0.0556 = 9.0350$
 - $\ast = 9.0350 + 0.0556 = 9.0906$
 - $\ast = 9.0906 + 0.0556 = 9.1462$
 - $\ast = 9.1462 + 0.0556 = 9.2018$
 - $\ast = 9.2018 + 0.0556 = 9.2574$
 - $\ast = 9.2574 + 0.0556 = 9.3130$
 - $\ast = 9.3130 + 0.0556 = 9.3686$
 - $\ast = 9.3686 + 0.0556 = 9.4242$
 - $\ast = 9.4242 + 0.0556 = 9.4798$
 - $\ast = 9.4798 + 0.0556 = 9.5354$
 - $\ast = 9.5354 + 0.0556 = 9.5910$
 - $\ast = 9.5910 + 0.0556 = 9.6466$
 - $\ast = 9.6466 + 0.0556 = 9.7022$
 - $\ast = 9.7022 + 0.0556 = 9.7578$
 - $\ast = 9.7578 + 0.0556 = 9.8134$
 - $\ast = 9.8134 + 0.0556 = 9.8690$
 - $\ast = 9.8690 + 0.0556 = 9.9246$
 - $\ast = 9.9246 + 0.0556 = 9.9802$
 - $\ast = 9.9802 + 0.0556 = 10.0358$
 - $\ast = 10.0358 + 0.0556 = 10.0914$
 - $\ast = 10.0914 + 0.0556 = 10.1470$
 - $\ast = 10.1470 + 0.0556 = 10.2026$
 - $\ast = 10.2026 + 0.0556 = 10.2582$
 - $\ast = 10.2582 + 0.0556 = 10.3138$
 - $\ast = 10.3138 + 0.0556 = 10.3694$
 - $\ast = 10.3694 + 0.0556 = 10.4250$
 - $\ast = 10.4250 + 0.0556 = 10.4806$
 - $\ast = 10.4806 + 0.0556 = 10.5362$
 - $\ast = 10.5362 + 0.0556 = 10.5918$
 - $\ast = 10.5918 + 0.0556 = 10.6474$
 - $\ast = 10.6474 + 0.0556 = 10.7030$
 - $\ast = 10.7030 + 0.0556 = 10.7586$
 - $\ast = 10.7586 + 0.0556 = 10.8142$
 - $\ast = 10.8142 + 0.0556 = 10.8698$
 - $\ast = 10.8698 + 0.0556 = 10.9254$
 - $\ast = 10.9254 + 0.0556 = 10.9810$
 - $\ast = 10.9810 + 0.0556 = 11.0366$
 - $\ast = 11.0366 + 0.0556 = 11.0922$
 - $\ast = 11.0922 + 0.0556 = 11.1478$
 - $\ast = 11.1478 + 0.0556 = 11.2034$
 - $\ast = 11.2034 + 0.0556 = 11.2590$
 - $\ast = 11.2590 + 0.0556 = 11.3146$
 - $\ast = 11.3146 + 0.0556 = 11.3702$
 - $\ast = 11.3702 + 0.0556 = 11.4258$
 - $\ast = 11.4258 + 0.0556 = 11.4814$
 - $\ast = 11.4814 + 0.0556 = 11.5370$
 - $\ast = 11.5370 + 0.0556 = 11.5926$
 - $\ast = 11.5926 + 0.0556 = 11.6482$
 - $\ast = 11.6482 + 0.0556 = 11.7038$
 - $\ast = 11.7038 + 0.0556 = 11.7594$
 - $\ast = 11.7594 + 0.0556 = 11.8150$
 - $\ast = 11.8150 + 0.0556 = 11.8706$
 - $\ast = 11.8706 + 0.0556 = 11.9262$
 - $\ast = 11.9262 + 0.0556 = 11.9818$
 - $\ast = 11.9818 + 0.0556 = 12.0374$
 - $\ast = 12.0374 + 0.0556 = 12.0930$
 - $\ast = 12.0930 + 0.0556 = 12.1486$
 - $\ast = 12.1486 + 0.0556 = 12.2042$
 - $\ast = 12.2042 + 0.0556 = 12.2598$
 - $\ast = 12.2598 + 0.0556 = 12.3154$
 - $\ast = 12.3154 + 0.0556 = 12.3710$
 - $\ast = 12.3710 + 0.0556 = 12.4266$
 - $\ast = 12.4266 + 0.0556 = 12.4822$
 - $\ast = 12.4822 + 0.0556 = 12.5378$
 - $\ast = 12.5378 + 0.0556 = 12.5934$
 - $\ast = 12.5934 + 0.0556 = 12.6490$
 - $\ast = 12.6490 + 0.0556 = 12.7046$
 - $\ast = 12.7046 + 0.0556 = 12.7602$
 - $\ast = 12.7602 + 0.0556 = 12.8158$
 - $\ast = 12.8158 + 0.0556 = 12.8714$
 - $\ast = 12.8714 + 0.0556 = 12.9270$
 - $\ast = 12.9270 + 0.0556 = 12.9826$
 - $\ast = 12.9826 + 0.0556 = 13.0382$
 - $\ast = 13.0382 + 0.0556 = 13.0938$
 - $\ast = 13.0938 + 0.0556 = 13.1494$
 - $\ast = 13.1494 + 0.0556 = 13.2050$
 - $\ast = 13.2050 + 0.0556 = 13.2606$
 - $\ast = 13.2606 + 0.0556 = 13.3162$
 - $\ast = 13.3162 + 0.0556 = 13.3718$
 - $\ast = 13.3718 + 0.0556 = 13.4274$
 - $\ast = 13.4274 + 0.0556 = 13.4830$
 - $\ast = 13.4830 + 0.0556 = 13.5386$
 - $\ast = 13.5386 + 0.0556 = 13.5942$
 - $\ast = 13.5942 + 0.0556 = 13.6498$
 - $\ast = 13.6498 + 0.0556 = 13.7054$
 - $\ast = 13.7054 + 0.0556 = 13.7610$
 - $\ast = 13.7610 + 0.0556 = 13.8166$
 - $\ast = 13.8166 + 0.0556 = 13.8722$
 - $\ast = 13.8722 + 0.0556 = 13.9278$
 - $\ast = 13.9278 + 0.0556 = 13.9834$
 - $\ast = 13.9834 + 0.0556 = 14.0390$
 - $\ast = 14.0390 + 0.0556 = 14.0946$
 - $\ast = 14.0946 + 0.0556 = 14.1502$
 - $\ast = 14.1502 + 0.0556 = 14.2058$
 - $\ast = 14.2058 + 0.0556 = 14.2614$
 - $\ast = 14.2614 + 0.0556 = 14.3170$
 - $\ast = 14.3170 + 0.0556 = 14.3726$
 - $\ast = 14.3726 + 0.0556 = 14.4282$
 - $\ast = 14.4282 + 0.0556 = 14.4838$
 - $\ast = 14.4838 + 0.0556 = 14.5394$
 - $\ast = 14.5394 + 0.0556 = 14.5950$
 - $\ast = 14.5950 + 0.0556 = 14.6506$
 - $\ast = 14.6506 + 0.0556 = 14.7062$
 - $\ast = 14.7062 + 0.0556 = 14.7618$
 - $\ast = 14.7618 + 0.0556 = 14.8174$
 - $\ast = 14.8174 + 0.0556 = 14.8730$
 - $\ast = 14.8730 + 0.0556 = 14.9286$
 - $\ast = 14.9286 + 0.0556 = 14.9842$
 - $\ast = 14.9842 + 0.0556 = 15.0398$
 - $\ast = 15.0398 + 0.0556 = 15.0954$
 - $\ast = 15.0954 + 0.0556 = 15.1510$
 - $\ast = 15.1510 + 0.0556 = 15.2066$
 - $\ast = 15.2066 + 0.0556 = 15.2622$
 - $\ast = 15.2622 + 0.0556 = 15.3178$
 - $\ast = 15.3178 + 0.0556 = 15.3734$
 - $\ast = 15.3734 + 0.0556 = 15.4290$
 - $\ast = 15.4290 + 0.0556 = 15.4846$
 - $\ast = 15.4846 + 0.0556 = 15.5402$
 - $\ast = 15.5402 + 0.0556 = 15.5958$
 - $\ast = 15.5958 + 0.0556 = 15.6514$
 - $\ast = 15.6514 + 0.0556 = 15.7070$
 - $\ast = 15.7070 + 0.0556 = 15.7626$
 - $\ast = 15.7626 + 0.0556 = 15.8182$
 - $\ast = 15.8182 + 0.0556 = 15.8738$
 - $\ast = 15.8738 + 0.0556 = 15.9294$
 - $\ast = 15.9294 + 0.0556 = 15.9850$
 - $\ast = 15.9850 + 0.0556 = 16.0406$
 - $\ast = 16.0406 + 0.0556 = 16.0962$
 - $\ast = 16.0962 + 0.0556 = 16.1518$
 - $\ast = 16.1518 + 0.0556 = 16.2074$
 - $\ast = 16.2074 + 0.0556 = 16.2630$
 - $\ast = 16.2630 + 0.0556 = 16.3186$
 - $\ast = 16.3186 + 0.0556 = 16.3742$
 - $\ast = 16.3742 + 0.0556 = 16.4298$
 - $\ast = 16.4298 + 0.0556 = 16.4854$
 - $\ast = 16.4854 + 0.0556 = 16.5410$
 - $\ast = 16.5410 + 0.0556 = 16.5966$
 - $\ast = 16.5966 + 0.0556 = 16.6522$
 - $\ast = 16.6522 + 0.0556 = 16.7078$
 - $\ast = 16.7078 + 0.0556 = 16.7634$
 - $\ast = 16.7634 + 0.0556 = 16.8190$
 - $\$

8 Discussion

This study demonstrates that compact CNNs within the MediaPipe/TFLite stack can learn posture-relevant features, but current discrimination is limited.

8.1 Interpretation of findings

The strongest three-class model (EfficientNet-Lite4) reached 41.2% accuracy (41.7% balanced accuracy). Reformulating the task as binary improved raw accuracy to 55.9% (55.6% balanced) with landmark overlays, indicating that explicit skeletal cues modestly aid detection. To compare the results of the 3-class and 2-class training, we can look at the overall accuracy and balanced accuracy (macro recall) for each model. The 3-class model achieved an overall accuracy of 41.21% and a balanced accuracy of 41.67%, while the 2-class model with landmark overlays achieved an overall accuracy of 55.85% and a balanced accuracy of 55.62%. Also the overall low performances of both CNN model training approaches (2-class as well as 3-class only slightly above chance) suggest that the task is challenging and that further improvements in data quality, model architecture, or training strategy may be needed to achieve clinically useful performance.

8.2 Strengths and limitations

The strongest three-class model (EfficientNet-Lite4) reached 41.2% accuracy (41.7% balanced accuracy). Reformulating the task as binary improved raw accuracy to 55.9% (55.6% balanced) with landmark overlays, indicating that explicit skeletal cues modestly aid detection. However, because the binary chance level is higher ($1/2$ vs $1/3$), these gains largely reflect problem simplification; after accounting for chance, both formulations perform only slightly above baseline. Persistent confusions—especially “normal” misclassified as “slumped”—suggest real class overlap, subtle posture cues, and probable label noise. Given the small dataset and static imagery, these findings point to feasibility rather than readiness for clinical deployment. Priorities for improvement include targeted data growth at class boundaries, refined and possibly multi-rater labels, posture-aware augmentation and sampling, and post-hoc calibration or thresholds to balance sensitivity and specificity for clinically meaningful use.

8.3 Clinical and practical implications

Given current performance (balanced accuracy 41% for three-class), the model’s most appropriate near-term role is as a clinician-in-the-loop decision support tool rather than a standalone diagnostic. Keeping in mind that $n=300$, further annotation and data expansion are needed to train the model to a better overall performance. In practice, the system can assist screening by flagging potentially “non-normal” cases for closer review, standardize documentation via landmark/angle overlays, and track an individual’s progress over time under consistent capture conditions. Safe deployment requires confidence calibration with an “abstain/needs review” state, threshold tuning to the intended use (e.g., sensitivity-oriented screening), and clear presentation of uncertainty. To enhance reliability and equity, future work should expand and re-balance labeled data—particularly at class boundaries—adopt multi-rater labeling, incorporate interpretable posture-aware features (e.g., craniovertebral angle), and validate externally across sites, devices, and demographic subgroups. On-device TFLite inference supports privacy and low latency, but any clinical use should be governed by explicit intended-use documentation, bias and robustness monitoring.

9 Conclusion and Future Work

This dissertation demonstrates that transfer learning on compact CNNs within the MediaPipe/TFLite stack is feasible for static upper-body posture assessment, but current performance does not yet meet clinical decision thresholds. The strongest three-class model (EfficientNet-Lite4) achieved 41.2% accuracy (41.7% balanced accuracy). Persistent confusions—particularly over-prediction of “slumped” and low recall for “normal”—likely reflect subtle class boundaries, label noise, and the small, static dataset. Practically, the present system is best positioned as clinician-in-the-loop decision support for screening, standardized documentation (landmark/angle overlays), and within-subject progress tracking under a consistent capture protocol. On-device TFLite supports privacy and low latency, but safe use requires calibrated confidence, explicit uncertainty, and clear guardrails around intended use.

Priorities for future work will center on prospective clinical image collection, expert annotation, and retraining the three-class model to improve discrimination and equity.

Clinical data collection: Acquire a larger, prospectively captured, side-view image set in real clinical settings under a standardized protocol (camera height = shoulder level, fixed distance, lateral stance, neutral footwear, uncluttered background, even lighting). Ensure representative demographics and obtain informed consent/ethics approvals; record capture metadata (device, view angle, lighting).

Expert annotation: Use multi-rater labeling (2 clinicians) with an explicit rubric aligned to the Klein-Vogelbach classes (“normal/slumped/tensed”) and adjudication for disagreements. Track inter-rater agreement (e.g., Cohen’s κ) and maintain a gold-standard subset for calibration and regression testing.

Retraining the 3-class model: Retrain with class-balanced sampling and posture-aware augmentation focused on boundary cases. Integrate interpretable posture features from landmarks (e.g., craniovertebral angle, thoracic kyphosis proxy) and consider a multi-task head (class + angles) to improve both accuracy and interpretability. Calibrate probabilities and set operating thresholds to match screening use.

Evaluation and fairness: Perform external validation on a held-out site/device; report subgroup performance (sex, age bands, skin tone) and confusion patterns. Use learning-curve analyses to quantify data needs and guide additional collection.

Deployment readiness: Keep inference on-device; implement confidence gating with “abstain/needs review,” version datasets/models, and monitor for drift post-deployment with periodic recalibration.

Milestones: Pilot n 1000 newly collected, class-balanced clinical images; target three-class balanced accuracy 0.60–0.65 on an external test split.

10 Appendix A. Figures

Figure 10: DatabaseData.png

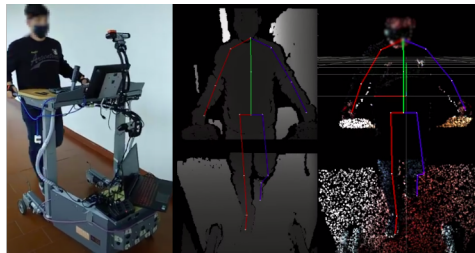


Fig A1: Smart Walker with the mobile recording setup; depth image overlaid with projected 2D skeleton; point-cloud overlaid with aligned 3D skeleton. **Source:** *Palermo et al. (2021)*

Figure 11: Timeline.png

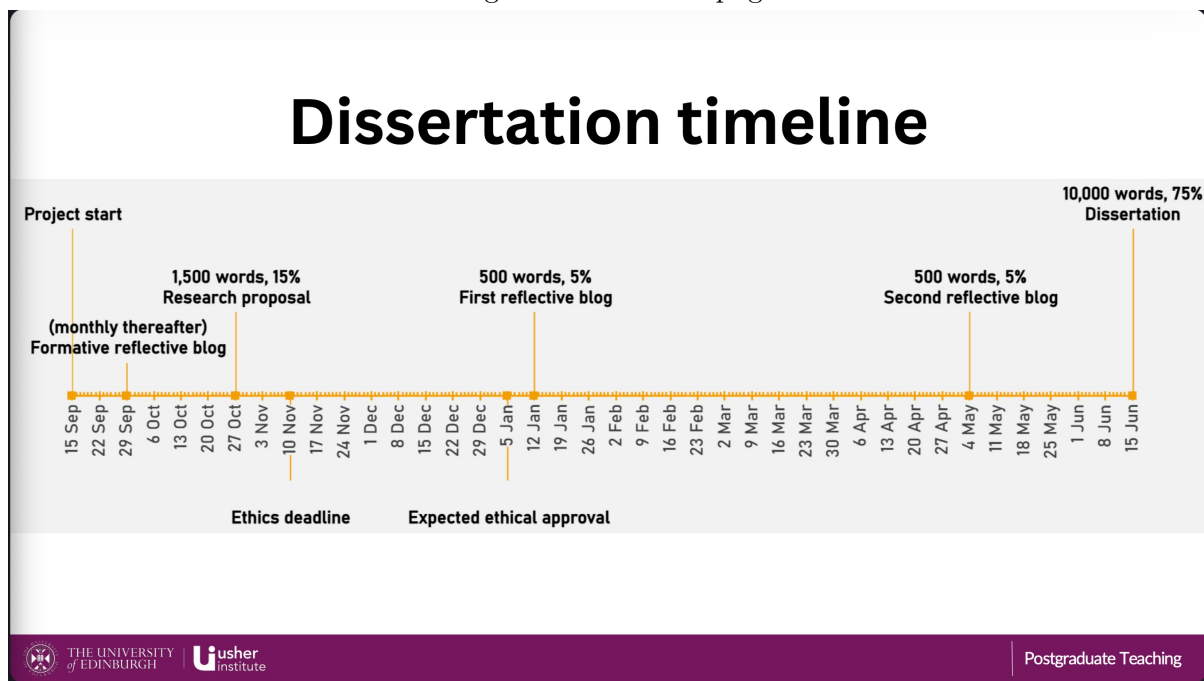


Fig A2: Project timeline outlining key milestones and deliverables. **Source:** *University of Edinburgh MSc in Data Science for Health and Social Care program*

References

- Badhe, P. C., & Kulkarni, V. (2018). A review on posture assessment. In <https://www.iosrjournals.org/iosr-jspe/papers/Vol-5Issue5/Version-1/B05050815.pdf>. https://www.researchgate.net/publication/328981577_A_Review_on_Posture_Assessment
- Bazarevsky, V., & Grishchenko, I. (2020). *On-device, real-time body pose tracking with MediaPipe BlazePose*. <https://research.google/blog/on-device-real-time-body-pose-tracking-with-mediapipe-blazepose/>
- Cao, X., Wang, X., Geng, X., Wu, D., & An, H. (2024). An approach for human posture recognition based on the fusion PSE-CNN-BiGRU model. *CMES - Computer Modeling in Engineering and Sciences*, 140, 385–408. <https://doi.org/10.32604/CMES.2024.046752>
- F.M. Alexander. (2017). Methode mit großem betriebsmedizinischem potential eigene, die berufsfähigkeit existentieller bedrohende probleme mit der stimme führten. *Institut Für Arbeits-, Sozial- Und Umweltmedizin*. <https://doi.org/10.2017>
- Ge, L., Pereira, M. J., Yap, C. W., & Heng, B. H. (2022). Chronic low back pain and its impact on physical function, mental health, and health-related quality of life: A cross-sectional study in singapore. *Scientific Reports*, 12. <https://doi.org/10.1038/S41598-022-24703-7>
- Google. (2024). *Pose detection | ML kit | google for developers*. <https://developers.google.com/ml-kit/vision/pose-detection>
- Jiang, X., Hu, Z., Wang, S., & Zhang, Y. (2023). A survey on artificial intelligence in posture recognition. *Computer Modeling in Engineering & Sciences*, 137, 35–82. <https://doi.org/10.32604/cmes.2023.027676>
- Kakuta, K. C., Kikawa, T., Kumagai, Y., Gupta, J., Pathak, S., & Kumar, G. (2022). Deep learning (CNN) and transfer learning: A review. *Journal of Physics: Conference Series*, 2273, 012029. <https://doi.org/10.1088/1742-6596/2273/1/012029>
- Klein-Vogelbach, S. (1990). Functional kinetics. *Functional Kinetics*. <https://doi.org/10.1007/978-3-642-95470-2>
- Kubalek-Schröder, S., & Dehler, F. (2013). Funktionsabhängige beschwerdebilder des bewegungssystems. *Funktionsabhängige Beschwerdebilder Des Bewegungssystems*. <https://doi.org/10.1007/978-3-642-35151-8>
- Lachance, J. M., Thong, W., Shruti, N., & Xiang, A. (2023). A case study in fairness evaluation: Current limitations and challenges for human pose estimation. *Sony*. <https://r2hcai.github.io/AAAI-23/files/CameraReadys/21.pdf>
- Move AI. (2026). Move AI - markerless motion capture & 3D animation. In <https://move.ai>. <https://move.ai/>
- RunDNA. (2026). Real-time 3D running gait analysis. <https://Rundna.com/>. <https://rundna.com/helix3d/>
- RUNRIGHT 3D. (2026). A running gait analysis and shoe fitting system. In <https://runright-3d.com>. <https://runright-3d.com/>
- Sahrmann, S., Azevedo, D. C., & Dillen, L. V. (2017). Diagnosis and treatment of movement system impairment syndromes. *Brazilian Journal of Physical Therapy*, 21, 391. <https://doi.org/10.1016/J.BJPT.2017.08.001>
- Tachihara, H., & Hamada, J. (2019). Characteristic movement of the ribs, thoracic vertebrae while elevating the upper limbs - influences of age and gender on movements. *The Open Orthopaedics Journal*, 13, 170–176. <https://doi.org/10.2174/1874325001913010170>
- TecnoBody Walker View. (2026). TecnoBody view 3.0 SCX treadmill. In <https://www.medical-xprt.com/products/tecnobody-model-view-30-scx-treadmill-804362>. <https://www.medical-xprt.com/products/tecnobody-model-view-30-scx-treadmill-804362>
- Ultralytics. (2025). Pose estimation - ultralytics YOLO docs. In *ultralytics*. <https://docs.ultralytics.com/tasks/pose/>
- ViMove2. (2026). ViMove2: Wearable sensor technology. In <https://runreborn.com/article/vi-move2>

move2-wearable-sensor-technology. <https://runreborn.com/article/vi-move2-wearable-sensor-technology>

- Wagh, V., Scott, M. W., & Kraeutner, S. N. (2024). Quantifying similarities between MediaPipe and a known standard to address issues in tracking 2D upper limb trajectories: Proof of concept study. *JMIR Formative Research*, 8, e56682. <https://doi.org/10.2196/56682>
- Whittaker, J. L., Booysen, N., Motte, S. D. L., Dennett, L., Lewis, C. L., Wilson, D., McKay, C., Warner, M., Padua, D., Emery, C. A., & Stokes, M. (2017). Predicting sport and occupational lower extremity injury risk through movement quality screening: A systematic review. *British Journal of Sports Medicine*, 51, 580–585. <https://doi.org/10.1136/BJSPORTS-2016-096760>
- Wijekulasuriya, G. A., Woods, C. T., Kittel, A., & Larkin, P. (2025). The development and content of movement quality assessments in athletic populations: A systematic review and multilevel meta-analysis. *Sports Medicine - Open*, 11, 7. <https://doi.org/10.1186/S40798-025-00813-0>
- Xu, H., Bazavan, E. G., Zafir, A., Freeman, W. T., Sukthankar, R., & Sminchisescu, C. (2020). GHUM GHUML: Generative 3D human shape and articulated pose models. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 6183–6192. <https://doi.org/10.1109/CVPR42600.2020.00622>
- Zaucker, P. (2010). *Ballkoordination bei schülern mit unterschiedlicher sporterfahrung*.